

Software Design Document (SDD)

Module	ADC Driver
Version	1.0.0
Team	Carlos Cayetano · Adrian Amaro · Adalberto Aguirre · Daniel López
Date	14/05/2026
Institution	CETYS Universidad

1. Introduction

1.1. Purpose

This document describes the design and architecture of the ADC driver module for the STM32F411RE. It is based on the requirements defined in the SRS and covers data structures, register-level algorithms, and control flow. The intended audience is any developer who needs to implement, test, or extend the driver.

1.2. Scope

The ADC driver is a low-level embedded software component that abstracts the ADC1 peripheral on the STM32F411RE Nucleo board. It hides the CMSIS register layout from higher-level modules and exposes a typed C API for configuring channels, triggering conversions, and reading 12-bit results. Lab Assignment 03 uses this driver through the Sensor module to map potentiometer readings (PA0, ADC1_IN0) to a PWM duty cycle output.

1.3. Definitions and Acronyms

Term	Definition
ADC	Analog-to-Digital Converter — hardware block that samples an analog voltage and produces a digital value.

ADC1	The single ADC peripheral present on the STM32F411RE.
APB2	Advanced Peripheral Bus 2 — the bus that clocks ADC1 on the STM32F4 family.
ADON	ADC ON bit (CR2 bit 0) — powers the ADC hardware on or off.
ARR	Auto-Reload Register — used by TIM driver; referenced here for context only.
CMSIS	Cortex Microcontroller Software Interface Standard — provides the C register-map headers (stm32f4xx.h).
CNT	Timer counter register — referenced here for context only.
CONT	Continuous mode bit (CR2 bit 1) — keeps the ADC converting without stopping.
CR1/CR2	ADC control registers 1 and 2 — configure resolution, scan mode, triggers, and conversion start.
DR	ADC Data Register — holds the 12-bit result of the last regular conversion.
EOC	End of Conversion flag (SR bit 1) — set by hardware when a regular conversion finishes.
FR	Functional Requirement — requirement traceability label from the SRS document.
HSI	High-Speed Internal oscillator — the 16 MHz default clock source on the STM32F411RE after reset.
JEOC	End of Injected Conversion flag (SR bit 2) — set when an injected channel conversion finishes.
JDR1-4	Injected Data Registers 1 through 4 — hold results for injected channel conversions.
JSQR	Injected Sequence Register — specifies the channel and length of the injected conversion sequence.

JSWSTART	Injected software start bit (CR2 bit 22) — triggers an injected conversion via software.
PA0	GPIO pin A0 on the STM32F411RE — connected to ADC1_IN0 and the potentiometer wiper.
SMPR1/2	Sample Time Registers — configure conversion time (in ADC clock cycles) per channel.
SQR1/3	Regular Sequence Registers — define the channel order and sequence length for regular conversions.
SWSTART	Software Start bit (CR2 bit 30) — triggers a regular conversion via software.
SRS	Software Requirements Specification — the requirements document this SDD traces to.

1.4. Requirements

The functional requirements this module must satisfy are specified in SRS-ADC_Driver. The table below maps each FR to the function that implements it.

Requirement ID	Description
FR-1	The driver shall enable the ADC1 peripheral clock via RCC_APB2ENR and configure PA0 as an analog input. Implemented by adc_init().
FR-2	The driver shall power on the ADC peripheral by setting the ADON bit in CR2. Implemented by adc_enableAdc().
FR-3	The driver shall configure a regular conversion channel by writing to SQR3 and the appropriate SMPR register. Implemented by adc_setChannel().
FR-4	The driver shall configure an injected conversion channel by writing to JSQR and the appropriate SMPR register. Implemented by adc_setInjectedChannel().
FR-5	The driver shall trigger a single regular conversion by clearing CONT and setting SWSTART. Implemented by adc_startSingleConversion().

FR-6	The driver shall start continuous regular conversions by setting CONT and SWSTART. Implemented by <code>adc_startContinuousConversion()</code> .
FR-7	The driver shall trigger an injected conversion by setting JSWSTART in CR2. Implemented by <code>adc_startInjectedConversion()</code> .
FR-8	The driver shall read the 12-bit result of a regular conversion by polling EOC and reading DR. Implemented by <code>adc_readData()</code> .
FR-9	The driver shall read the 12-bit result of an injected conversion by polling JEOC and selecting the correct JDR register. Implemented by <code>adc_readInjectedChannelData()</code> .

2. System Architecture

2.1. High-Level Design

The software is organized in four layers. The application layer (main.c) drives the control loop without accessing hardware registers directly. The sensor layer translates application-level read requests into ADC driver calls. The ADC driver layer owns all register manipulation for ADC1. Below that, CMSIS provides the C struct definitions that map to the physical register addresses on the Cortex-M4.

- Application Layer: main.c calls `sensor_init()`, `sensor_startConversion()`, and `sensor_readValue()` — no ADC register knowledge required at this level.
- Sensor Layer (sensor.c): wraps the ADC driver and stores the active configuration in a static variable so the call site stays clean.
- ADC Driver Layer (adc_driver.c): enables clocks, configures PA0 MODER, sets SMPR/SQR/JSQR, starts conversions, and polls status flags.
- Hardware Layer: STM32F411RE ADC1 peripheral registers as defined in `stm32f411xe.h`.

2.2. Module Interfaces

The public API is declared in `adc_driver.h` and grouped into four functional areas: initialization (`adc_init`, `adc_enableAdc`), channel configuration (`adc_setChannel`, `adc_setInjectedChannel`), conversion trigger (`adc_startSingleConversion`, `adc_startContinuousConversion`, `adc_startInjectedConversion`), and result retrieval (`adc_readData`, `adc_readInjectedChannelData`).

The driver depends on the CMSIS header `stm32f4xx.h` for the `ADC_TypeDef` and `RCC_TypeDef` register structs, and includes `gpio_driver.h` for the GPIOA clock enable macro. To avoid resetting the PA6 alternate function already configured by the PWM module, `adc_init()` writes directly to `GPIOA->MODER` instead of calling `gpio_initPort()`.

Key CMSIS register references used by this driver: `ADC1->CR1`, `ADC1->CR2`, `ADC1->SQR1`, `ADC1->SQR3`, `ADC1->SMPR1`, `ADC1->SMPR2`, `ADC1->JSQR`, `ADC1->JDR1-4`, `ADC1->DR`, `ADC1->SR`, `RCC->APB2ENR`, `RCC->AHB1ENR`.

3. Detailed Design

3.1. Class Diagram

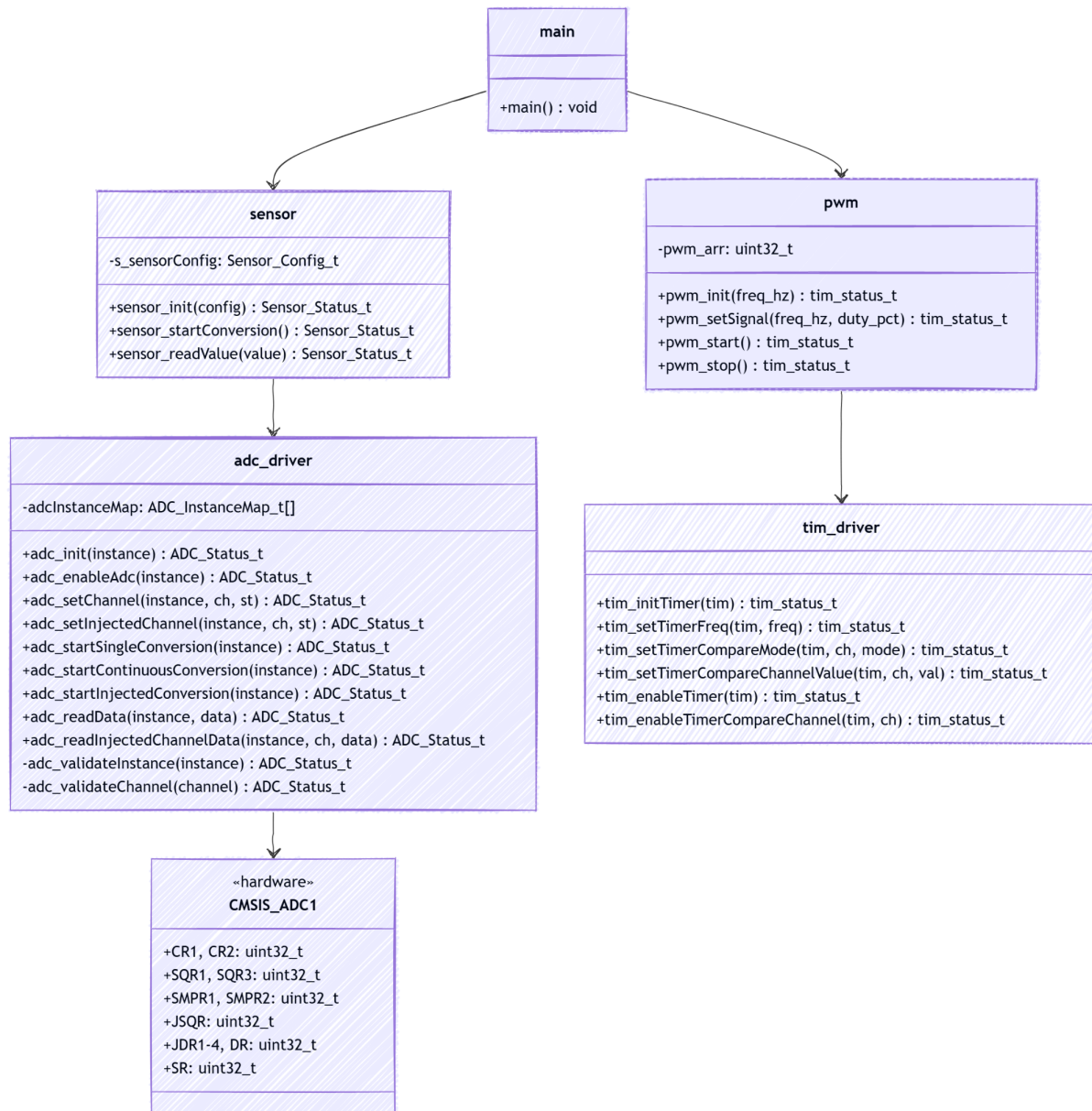


Figure 1. Class Diagram — ADC Driver

3.2. Module Behavior (Sequence Diagrams, State Machines, etc.)

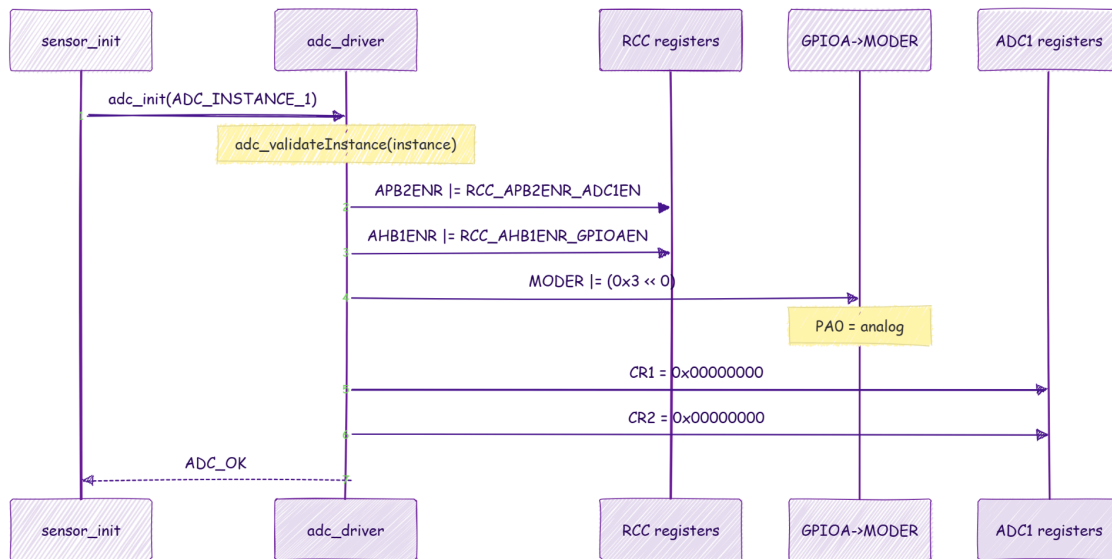


Figure 2. Sequence Diagram — adc_init

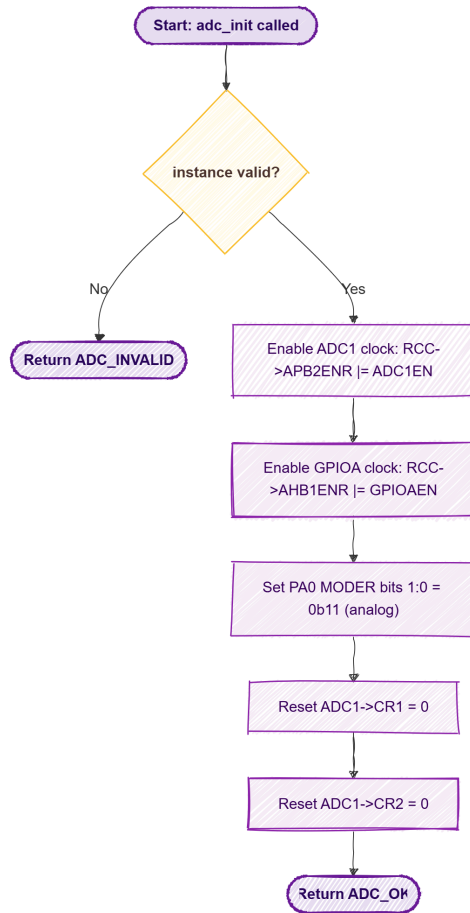


Figure 3. Activity Diagram — `adc_init`

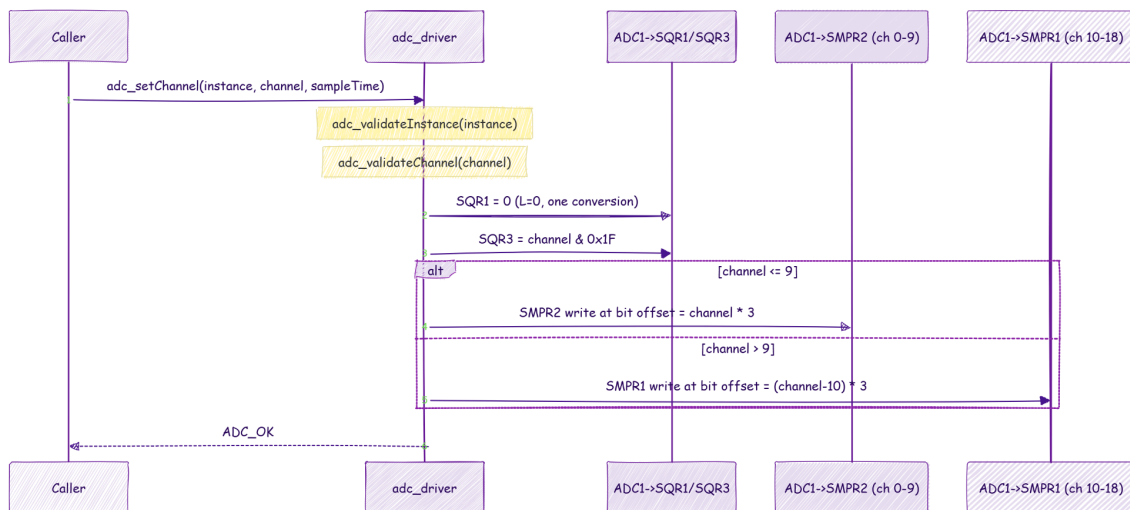


Figure 4. Sequence Diagram — `adc_setChannel`

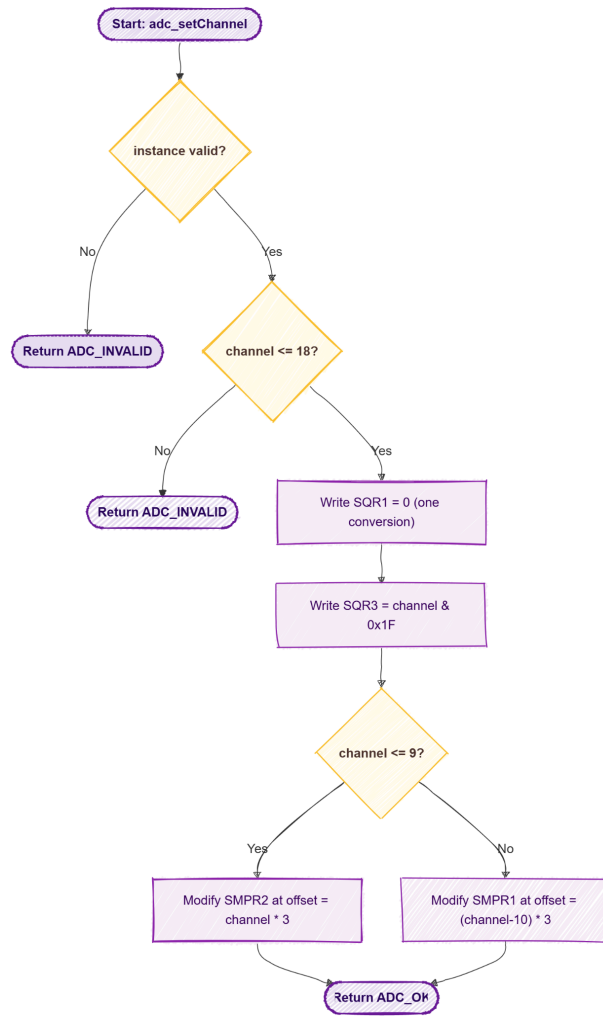


Figure 5. Activity Diagram — adc_setChannel

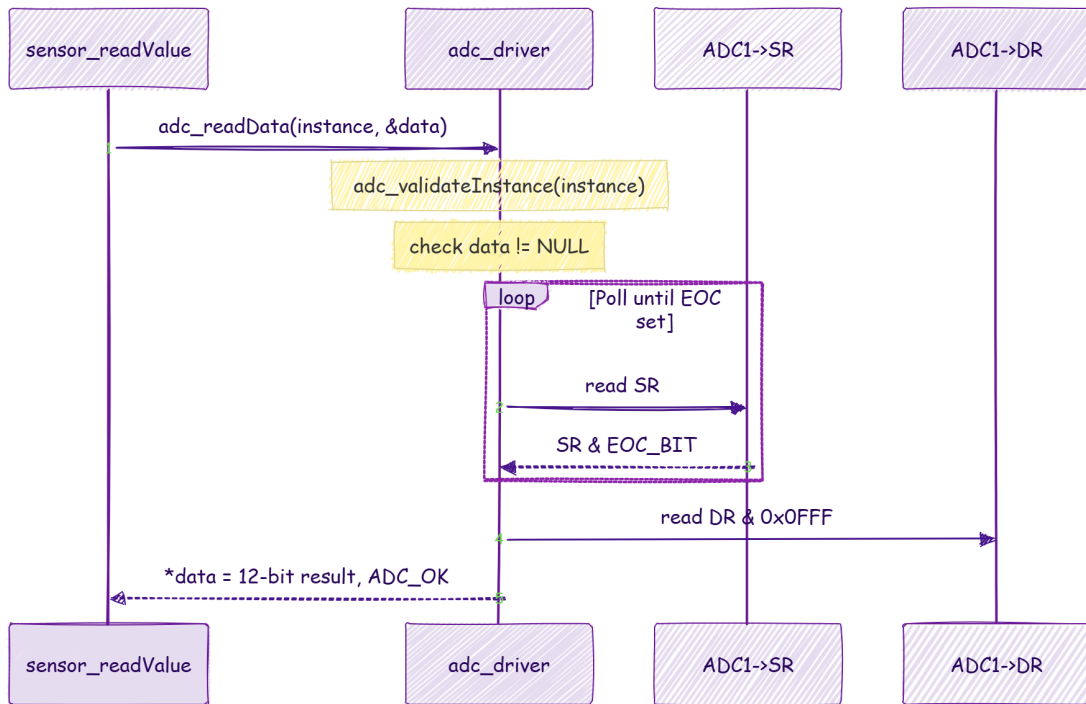


Figure 6. Sequence Diagram — `adc_readData`

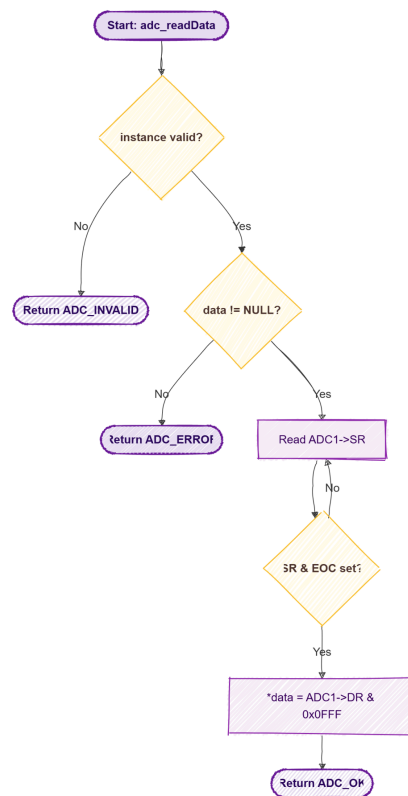


Figure 7. Activity Diagram — `adc_readData`

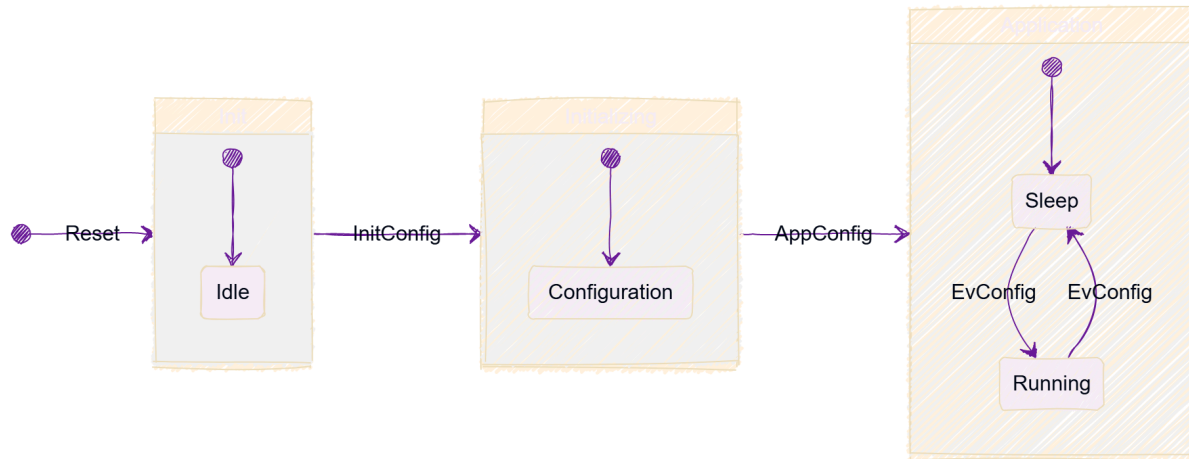


Figure 8. State Diagram — ADC Driver

3.3. ADC Driver

The driver is organized around a single private lookup table, `adcInstanceMap[]`, which maps each `ADC_Instance_t` enum value to its hardware register pointer and RCC clock enable bit. All public functions start by calling `adc_validateInstance()` and, where applicable, `adc_validateChannel()` before touching any register. This keeps error handling consistent and avoids duplicating guard logic across nine entry points.

Initialization

Initialization (`adc_init`) runs four writes in sequence: it enables the ADC1 APB2 clock, enables the GPIOA AHB1 clock, sets PA0 MODER bits to 0b11 (analog mode), then zeros CR1 and CR2. The PA0 configuration intentionally bypasses `gpio_initPort()` because that function resets the entire MODER register, which would destroy the PA6 alternate function already set up by `pwm_init()`. Writing two bits directly to GPIOA->MODER is the safer approach when two modules share the same GPIO port.

Configuration

Channel configuration (`adc_setChannel`) sets SQR1 to zero to specify a single-entry sequence, writes the channel number into SQR3, then programs the sample time into SMPR2 for channels 0-9 or SMPR1 for channels 10-18. Each SMPR field is three bits wide; the bit offset is `channel * 3` for SMPR2 and `(channel - 10) * 3` for SMPR1. The driver reads, masks, and writes the field to preserve settings for other channels that may be configured elsewhere.

Starting conversions is split into three functions because the CR2 bit pattern differs. Single conversion (`adc_startSingleConversion`) clears CONT then sets SWSTART.

Continuous mode (`adc_startContinuousConversion`) sets both `CONT` and `SWSTART` so the ADC restarts automatically after each result. Injected conversion (`adc_startInjectedConversion`) sets `JSWSTART` instead of `SWSTART`.

Reading a regular result (`adc_readData`) busy-waits on the `EOC` flag in `SR`, then reads the lower 12 bits of `DR`. The busy-wait is intentional for a bare-metal environment with no RTOS. For injected results, `adc_readInjectedChannelData` polls `JEOC` and selects `JDR1` through `JDR4` with a switch statement based on the injected channel number (1-4).

In Lab Assignment 03, the application calls functions in this order: `pwm_init()` first (to configure `PA6` before `GPIOA` is touched again), then `sensor_init()` which calls `adc_init()` + `adc_enableAdc()` + `adc_setChannel()` in sequence. Inside the main loop, `sensor_startConversion()` fires `adc_startSingleConversion()`, and `sensor_readValue()` calls `adc_readData()`. The result maps directly to a CCR value via $CCR = (ARR + 1) * adcValue / 4095$.

4. Application Programming Interfaces

4.1. Data Types and Macros

ADC_Status_t: return type for all public ADC driver functions. Three values: ADC_OK (0) — operation succeeded; ADC_ERROR (1) — generic failure, typically a NULL pointer; ADC_INVALID (2) — an argument was out of range.

ADC_Instance_t: identifies which ADC peripheral to use. The STM32F411RE has only one ADC, so this enum currently has a single member: ADC_INSTANCE_1 (0).

ADC_SampleTime_t: selects the number of ADC clock cycles to hold the sample. Higher counts improve accuracy for high-impedance sources like a potentiometer.

Value	Cycles
ADC_SAMPLETIME_3CYCLES	3 — fastest, lowest accuracy
ADC_SAMPLETIME_15CYCLES	15
ADC_SAMPLETIME_28CYCLES	28
ADC_SAMPLETIME_56CYCLES	56
ADC_SAMPLETIME_84CYCLES	84 — recommended for potentiometer on PA0
ADC_SAMPLETIME_112CYCLES	112
ADC_SAMPLETIME_144CYCLES	144
ADC_SAMPLETIME_480CYCLES	480 — slowest, highest accuracy

ADC_InstanceMap_t (private): maps an ADC_Instance_t value to a hardware register pointer (ADC_TypeDef*reg) and the RCC clock enable bit (uint32_t rccBit). Declared in adc_driver.c only.

Private macros defined in adc_driver.c: ADC_MAX_CHANNEL = 18 (highest valid channel index); ADC_MAX_INJECTED_CHANNEL = 4 (max injected channels); ADC_CR2_ADON_BIT = (1U << 0); ADC_CR2_CONT_BIT = (1U << 1); ADC_CR2_SWSTART_BIT = (1U << 30); ADC_CR2_JSWSTART_BIT = (1U << 22); ADC_SR_EOC_BIT = (1U << 1); ADC_SR_JEOC_BIT = (1U << 2); RCC_APB2ENR_ADC1EN_BIT = (1U << 8).

4.2 Error Codes

All ADC driver functions return ADC_Status_t. The three codes are described below.

Code	Definition
ADC_OK (0)	The operation completed without errors.
ADC_ERROR (1)	A generic failure occurred — typically a NULL output pointer passed to adc_readData() or adc_readInjectedChannelData().
ADC_INVALID (2)	An argument was out of the valid range: unrecognized instance, channel number above 18, or injected channel number outside 1-4.

4.3 Function Definitions

adc_init

Function Name	adc_init
Syntax	ADC_Status_t adc_init(ADC_Instance_t instance)
Parameters (in)	instance — ADC_Instance_t: the ADC peripheral to initialize (ADC_INSTANCE_1).
Parameters (inout)	None
Parameters (out)	None
Return value	ADC_Status_t — ADC_OK on success, ADC_INVALID if instance is not recognized.

Description	Enables the ADC1 APB2 clock, enables the GPIOA AHB1 clock, sets PA0 to analog input mode, and resets CR1 and CR2 to zero.
Constraints	Writes directly to GPIOA->MODER to avoid resetting PA6 alternate function set by the PWM module.
Available via	adc_driver.h

adc_enableAdc

Function Name	adc_enableAdc
Syntax	ADC_Status_t adc_enableAdc(ADC_Instance_t instance)
Parameters (in)	instance — ADC_Instance_t.
Parameters (inout)	None
Parameters (out)	None
Return value	ADC_Status_t — ADC_OK on success, ADC_INVALID if instance is unknown.
Description	Powers on the ADC peripheral by setting the ADON bit (bit 0) in CR2. Must be called after adc_init() and before any conversion.
Constraints	None.
Available via	adc_driver.h

adc_setChannel

Function Name	adc_setChannel
----------------------	----------------

Syntax	ADC_Status_t adc_setChannel(ADC_Instance_t instance, uint8_t channel, ADC_SampleTime_t sampleTime)
Parameters (in)	instance — ADC_Instance_t; channel — channel number 0-18; sampleTime — ADC_SampleTime_t.
Parameters (inout)	None
Parameters (out)	None
Return value	ADC_Status_t — ADC_OK, or ADC_INVALID if instance or channel is out of range.
Description	Configures a regular conversion: sets SQR1 = 0 (single-entry sequence), writes channel to SQR3, and programs the sampling time in SMPR2 (channels 0-9) or SMPR1 (channels 10-18).
Constraints	Channels above 18 are rejected. Overwrites any previous channel configuration in SQR1/SQR3.
Available via	adc_driver.h

adc_setInjectedChannel

Function Name	adc_setInjectedChannel
Syntax	ADC_Status_t adc_setInjectedChannel(ADC_Instance_t instance, uint8_t channel, ADC_SampleTime_t sampleTime)
Parameters (in)	instance — ADC_Instance_t; channel — 0-18; sampleTime — ADC_SampleTime_t.
Parameters (inout)	None
Parameters (out)	None
Return value	ADC_Status_t — ADC_OK or ADC_INVALID.
Description	Configures an injected conversion by writing the channel to JSQR (JSQ4 field, bits 14:10) with a sequence length of 1.

	Sample time is written to SMPR2 or SMPR1 using the same offset logic as <code>adc_setChannel</code> .
Constraints	Only one injected channel at a time is supported. Resets JSQR on each call.
Available via	<code>adc_driver.h</code>

adc_startSingleConversion

Function Name	<code>adc_startSingleConversion</code>
Syntax	<code>ADC_Status_t adc_startSingleConversion(ADC_Instance_t instance)</code>
Parameters (in)	<code>instance</code> — <code>ADC_Instance_t</code> .
Parameters (inout)	None
Parameters (out)	None
Return value	<code>ADC_Status_t</code> — <code>ADC_OK</code> or <code>ADC_INVALID</code> .
Description	Clears the CONT bit then sets SWSTART in CR2. The ADC performs one conversion and stops. Use <code>adc_readData()</code> to retrieve the result.
Constraints	The ADC must be enabled (<code>adc_enableAdc</code> called) and a channel configured (<code>adc_setChannel</code> called) before calling this function.
Available via	<code>adc_driver.h</code>

adc_startContinuousConversion

Function Name	<code>adc_startContinuousConversion</code>
Syntax	<code>ADC_Status_t adc_startContinuousConversion(ADC_Instance_t instance)</code>
Parameters (in)	<code>instance</code> — <code>ADC_Instance_t</code> .

Parameters (inout)	None
Parameters (out)	None
Return value	ADC_Status_t — ADC_OK or ADC_INVALID.
Description	Sets CONT and SWSTART in CR2. The ADC converts the configured channel repeatedly without stopping. Call adc_readData() in a loop to read each result as EOC toggles.
Constraints	There is no API call to stop continuous conversion in the current implementation. Stopping requires clearing SWSTART and CONT manually.
Available via	adc_driver.h

adc_startInjectedConversion

Function Name	adc_startInjectedConversion
Syntax	ADC_Status_t adc_startInjectedConversion(ADC_Instance_t instance)
Parameters (in)	instance — ADC_Instance_t.
Parameters (inout)	None
Parameters (out)	None
Return value	ADC_Status_t — ADC_OK or ADC_INVALID.
Description	Sets the JSWSTART bit in CR2 to trigger a software-initiated injected conversion. Use adc_readInjectedChannelData() to read the result.
Constraints	An injected channel must be configured with adc_setInjectedChannel() before calling this.
Available via	adc_driver.h

adc_readData

Function Name	adc_readData
Syntax	ADC_Status_t adc_readData(ADC_Instance_t instance, uint16_t *data)
Parameters (in)	instance — ADC_Instance_t.
Parameters (inout)	None
Parameters (out)	data — uint16_t*: output pointer where the 12-bit ADC result (0-4095) is written.
Return value	ADC_Status_t — ADC_OK on success, ADC_INVALID if instance unknown, ADC_ERROR if data is NULL.
Description	Busy-waits until the EOC flag in SR is set, then reads the lower 12 bits of DR into *data. Clears EOC by reading DR.
Constraints	Blocks indefinitely if no conversion was started before calling this function.
Available via	adc_driver.h

adc_readInjectedChannelData

Function Name	adc_readInjectedChannelData
Syntax	ADC_Status_t adc_readInjectedChannelData(ADC_Instance_t instance, uint8_t channel, uint16_t *data)
Parameters (in)	instance — ADC_Instance_t; channel — injected channel number 1-4.
Parameters (inout)	None
Parameters (out)	data — uint16_t*: output pointer for the 12-bit result.
Return value	ADC_Status_t — ADC_OK, ADC_INVALID if instance or channel is out of range, ADC_ERROR if data is NULL.
Description	Busy-waits on the JEOC flag in SR, then reads from JDR1, JDR2, JDR3, or JDR4 depending on the channel argument.

Constraints	Channel argument is 1-based (1-4), not 0-based. Passing 0 returns ADC_INVALID.
Available via	adc_driver.h

4.4 Internal Function Definitions

adc_validateInstance

Function Name	adc_validateInstance
Syntax	static ADC_Status_t adc_validateInstance(ADC_Instance_t instance)
Parameters (in)	instance — ADC_Instance_t value to validate.
Parameters (inout)	None
Parameters (out)	None
Return value	ADC_Status_t — ADC_OK if instance <= ADC_INSTANCE_1, ADC_INVALID otherwise.
Description	Checks that the instance argument does not exceed the highest valid enum value. Called at the start of every public function.
Constraints	Not exposed in adc_driver.h. Used only within adc_driver.c.

adc_validateChannel

Function Name	adc_validateChannel
Syntax	static ADC_Status_t adc_validateChannel(uint8_t channel)
Parameters (in)	channel — channel number to validate (0-18).
Parameters (inout)	None
Parameters (out)	None

Return value	ADC_Status_t — ADC_OK if channel <= ADC_MAX_CHANNEL (18), ADC_INVALID otherwise.
Description	Rejects any channel number above 18 before any register is written. Called by adc_setChannel() and adc_setInjectedChannel().
Constraints	Not exposed in adc_driver.h.

5. Future Enhancements

- DMA transfers: connect ADC DR directly to a memory buffer via DMA2 Stream 0/4 to remove the polling loop from adc_readData() and allow other work during conversion.
- Interrupt-driven EOC: use the ADC end-of-conversion interrupt (EOC interrupt enable in CR1) instead of busy-waiting, which would allow the CPU to sleep between samples.
- Multi-channel scan mode: extend the SQR1/SQR2/SQR3 configuration to support sequences of up to 16 regular channels, useful for reading multiple sensors in one sweep.
- Resolution configuration: expose CR1 RES bits (bits 25:24) to allow 10-bit, 8-bit, or 6-bit conversion for faster throughput when full 12-bit precision is not needed.
- Hardware trigger support: add an adc_setTrigger() function to configure EXTEN and EXTSEL in CR2, allowing timers or external pins to trigger conversions without software intervention.

- Continuous stop function: add `adc_stopContinuousConversion()` to clear CONT and SWSTART cleanly, giving application code a way to halt continuous mode without resetting the whole peripheral.

6. References

- [1] Gbati, O. (2023). Embedded Systems Programming with the STM32 Microcontroller. Independently published.
- [2] Mazidi, M., Chen, S., & Naimi, S. (2020). The STM32F0 ARM Microcontroller and Embedded Systems: Using Assembly and C. Microdigitaled.
- [3] Deitel, P., & Deitel, H. (2016). C How to Program (8th ed.). Pearson.
- [4] STMicroelectronics. (2016). RM0383: STM32F411 reference manual (Rev 3). https://www.st.com/resource/en/reference_manual/rm0383-stm32f411xce-advanced-armbased-32bit-mcus-stmicroelectronics.pdf

